STAT3888: Model Diagnostics and Evaluation

AUTHOR
Calvin Lai

PUBLISHED
September 4, 2026

In this document, we will try to conduct some model diagnostics and evaluation for our four multinomial logistic regression models. In particular, we will check

1. Model Diagnostics: Convergence, sparsity, outliers, and assumption checking.
2. Sensitivity Analysis: Influence (using case-deletion sensitivity).
3. Model Evaluation: Log-loss and multi-class AUC; these will likely be between the models fit with `pdat.full` against those fit with `fdat.full` for ALT and GGT respectively.

▼ Code

```
# Initialise data (only complete cases, since our model is trai
# complete cases only anyway)
pdat.full = readRDS(here("Objects", "pdat_full.rds"))
fdat.full = readRDS(here("Objects", "fdat_full.rds"))

# Initialise models
alt.mlr = readRDS(here("Objects", "MLR_ALT3.rds"))
ggt.mlr = readRDS(here("Objects", "MLR_GGT3.rds"))
alt.fmlr = readRDS(here("Objects", "Full_data_MLR_ALT3.rds"))
ggt.fmlr = readRDS(here("Objects", "Full_data_MLR_GGT3.rds"))
```

Model Diagnostics

Sparsity Checks

We first create contingency tables for each variable/outcome pair and model, for potential sparsity.

▼ Code

```
# Contingency Tables for alt.mlr
xtabs(~ ALT3 + AGEB, data = model.frame(alt.mlr))
```

```
            AGEB
ALT3         18–19 years 20–24 years 25–29 years 30–34 years
35–39 years
  Low                  0           2          22          20
24
  Borderline           0           2           5           2
0
  High                 0           1           0           2
2
            AGEB
ALT3         40–44 years 45–49 years 50–54 years 55–59 years
60–64 years
  Low                 32          37          40          38
28
  Borderline           6           6           5           6
10
  High                 9          10           6          12
17
            AGEB
ALT3         65–69 years 70–74 years 75–79 years 80–84 years 85
years and older
  Low                 27          28          18          12
4
  Borderline           5           3           1           1
0
  High                12           3           1           1
0
```

▼ Code

```
xtabs(~ ALT3 + SEX, data = model.frame(alt.mlr))
```

```
            SEX
ALT3         Female Male
  Low           181  151
  Borderline     15   37
  High           24   52
```

▼ Code

```
xtabs(~ ALT3 + DSMKHHRB, data = model.frame(alt.mlr))
```

```
            DSMKHHRB
ALT3         None One Two Three or more
  Low         212  93  25              2
  Borderline   39  10   3              0
  High         51  18   6              1
```

▼ Code

```
xtabs(~ ALT3 + WAIST, data = model.frame(alt.mlr))
```

```
           WAIST
ALT3        Increased risk Not at risk Substantially increased
risk
  Low                   71         117
144
  Borderline            11          11
30
  High                  25           9
42
```

▼ Code

```
# Contingency Tables for ggt.mlr
xtabs(~ GGT3 + AGEB, data = model.frame(ggt.mlr))
```

```
           AGEB
GGT3        18–19 years 20–24 years 25–29 years 30–34 years
35–39 years
  Low                 0           5          22          19
22
  Borderline          0           0           3           4
1
  High                0           0           2           1
3
           AGEB
GGT3        40–44 years 45–49 years 50–54 years 55–59 years
60–64 years
  Low                31          35          30          40
30
  Borderline          6           7           6           3
1
  High               10          11          15          13
24
           AGEB
GGT3        65–69 years 70–74 years 75–79 years 80–84 years 85
years and older
  Low                24          22          14           9
2
  Borderline          4           3           0           2
1
  High               16           9           6           3
1
```

▼ Code

```
xtabs(~ GGT3 + SEX, data = model.frame(ggt.mlr))
```

```
           SEX
GGT3        Female Male
  Low          159  146
  Borderline    19   22
  High          42   72
```

▼ Code

```
xtabs(~ GGT3 + DSMKHHRB, data = model.frame(ggt.mlr))
```

```
           DSMKHHRB
GGT3        None One Two Three or more
  Low        200  82  20             3
  Borderline  27  10   4             0
  High        75  29  10             0
```

▼ Code

```
xtabs(~ GGT3 + WAIST, data = model.frame(ggt.mlr))
```

```
           WAIST
GGT3        Increased risk Not at risk Substantially increased
risk
  Low                   73         101
131
  Borderline             9          11
21
  High                  25          25
64
```

▼ Code

```
# Contingency Tables for alt.fmlr
xtabs(~ ALT3 + AGEB, data = model.frame(alt.fmlr))
```

```
           AGEB
ALT3        18-19 years 20-24 years 25-29 years 30-34 years
35-39 years
  Low                56         141         267         272
319
  Borderline          4          11          41          38
32
```

```
  High                 5          15          36          38
66
            AGEB
ALT3         40–44 years 45–49 years 50–54 years 55–59 years
60–64 years
  Low                379         380         382         409
445
  Borderline          48          75          79          63
96
  High                63          84          84          74
82
            AGEB
ALT3         65–69 years 70–74 years 75–79 years 80–84 years 85
years and older
  Low                394         272         225         155
76
  Borderline          51          38           9           9
1
  High                57          26          11           1
1
```

▼ Code

```
xtabs(~ ALT3 + SEX, data = model.frame(alt.fmlr))
```

```
            SEX
ALT3         Female Male
  Low          2446 1726
  Borderline    221  374
  High          283  360
```

▼ Code

```
xtabs(~ ALT3 + DSMKHHRB, data = model.frame(alt.fmlr))
```

```
            DSMKHHRB
ALT3         None  One  Two Three or more
  Low        3399  606  156            11
  Borderline  499   78   15             3
  High        519   96   26             2
```

▼ Code

```
xtabs(~ ALT3 + WAIST, data = model.frame(alt.fmlr))
```

```
            WAIST
```

```
ALT3         Increased risk Not at risk Substantially increased
risk
  Low                   980        1457
1735
  Borderline            140         127
328
  High                  128          88
427
```

▼ Code

```
# Contingency Tables for ggt.fmlr
xtabs(~ GGT3 + AGEB, data = model.frame(ggt.fmlr))
```

```
           AGEB
GGT3         18–19 years 20–24 years 25–29 years 30–34 years
35–39 years
  Low                 64         155         311         302
360
  Borderline           0           7          16          16
21
  High                 1           5          17          30
36
           AGEB
GGT3         40–44 years 45–49 years 50–54 years 55–59 years
60–64 years
  Low                411         437         418         430
467
  Borderline          25          33          34          34
42
  High                54          69          93          82
114
           AGEB
GGT3         65–69 years 70–74 years 75–79 years 80–84 years 85
years and older
  Low                387         266         196         133
61
  Borderline          26          20          12          11
3
  High                89          50          37          21
14
```

▼ Code

```
xtabs(~ GGT3 + SEX, data = model.frame(ggt.fmlr))
```

```
           SEX
```

```
GGT3         Female Male
  Low          2401 1997
  Borderline    145  155
  High          404  308
```

▼ Code

```
xtabs(~ GGT3 + DSMKHHRB, data = model.frame(ggt.fmlr))
```

```
           DSMKHHRB
GGT3         None  One  Two Three or more
  Low        3637  603  146            12
  Borderline  224   62   14             0
  High        556  115   37             4
```

▼ Code

```
xtabs(~ GGT3 + WAIST, data = model.frame(ggt.fmlr))
```

```
           WAIST
GGT3         Increased risk Not at risk Substantially increased
risk
  Low                 1036        1517
1845
  Borderline            73          50
177
  High                 139         105
468
```

Based on this, it may suggest at some point that we should collapse `DSMKHHRB` and `AGEB` levels.

Convergence, AIC and Deviance

Next, we check convergence of the models. If the model `convergence` value is `0` then it means the model iterations finished before hitting the maximum iteration limit; i.e. it converges successfully! We also summarise the deviance and AIC for each model.

▼ Code

```
# Convergence
conv = c(alt.mlr$convergence, ggt.mlr$convergence,
         alt.fmlr$convergence, ggt.fmlr$convergence)
conv.log = (conv == 0)
```

```
models = c("ALT MLR", "GGT MLR", "ALT Full MLR", "GGT Full MLR"
aic = c(alt.mlr$AIC, ggt.mlr$AIC, alt.fmlr$AIC, ggt.fmlr$AIC)
deviance = c(alt.mlr$deviance, ggt.mlr$deviance,
             alt.fmlr$deviance, ggt.fmlr$deviance)
models.df = data.frame("Model" = models, "Convergence" = conv.l
                       "AIC" = aic, "Deviance" = deviance)
kable(models.df, align = "cccc")
```

Model	Convergence	AIC	Deviance
ALT MLR	TRUE	691.9362	607.9362
GGT MLR	TRUE	784.0450	700.0450
ALT Full MLR	TRUE	7030.8105	6942.8105
GGT Full MLR	TRUE	6202.4844	6114.4844

Outliers

We use the probability assigned to the observed class, as a measure of outlyingness for each observation. We can do this by quantifying the individual deviance contribution, as implemented below.

▼ Code

```
# Outlier analysis for ALT Subpopulation Model
prob = predict(alt.mlr, type = "probs")
obs = model.response(model.frame(alt.mlr))
p_true = prob[cbind(seq_along(obs), match(obs, colnames(prob)))
dev_i = -2 * log(p_true)

# Top 10 observations
top.10.idx.alt = order(dev_i, decreasing = TRUE)[1:10]
top.10.devs.alt = dev_i[top.10.idx.alt]
top.10.outliers.alt = data.frame("ABSLID" = pdat.full$ABSLID[to
                                 "Deviance" = top.10.devs.alt)
kable(top.10.outliers.alt, align = "cc", caption = "ALT Model T
```

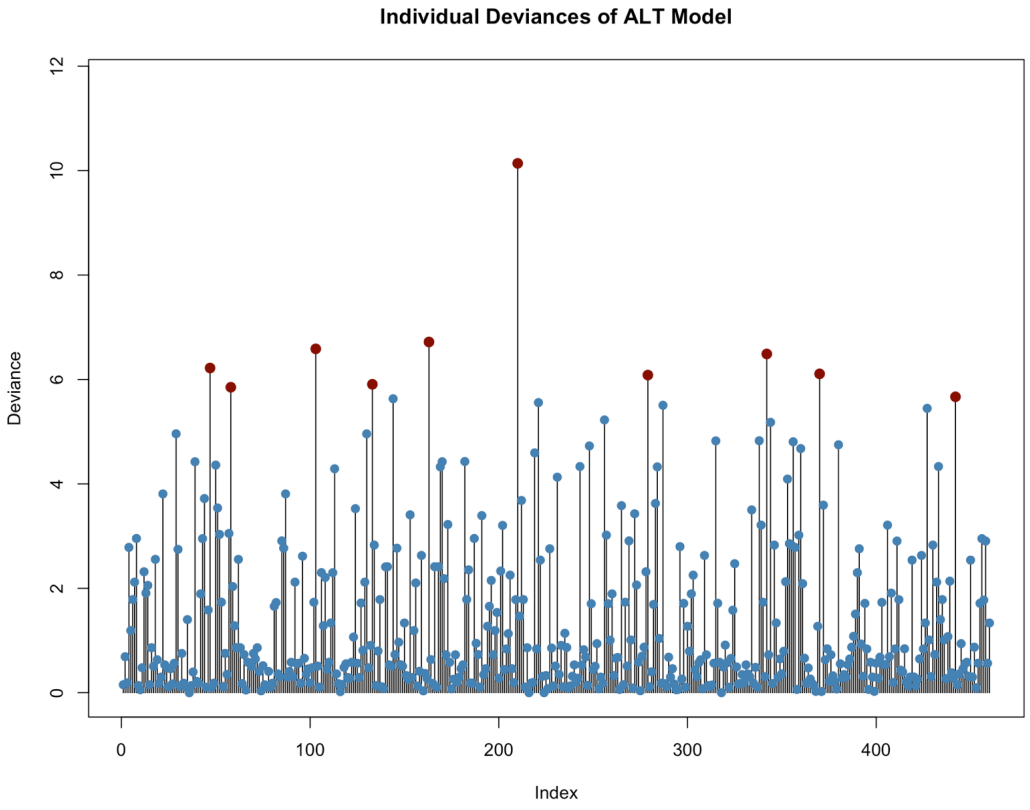
ALT Model Top 10 Outliers

ABSLID	Deviance
NHS11B0002374	10.139631
NHS11B0001941	6.718201

NHS11B0001120	6.586214
NHS11B0003836	6.488690
NHS11B0000549	6.219572
NHS11B0004160	6.107583
NHS11B0003157	6.084571
NHS11B0001523	5.907339
NHS11B0000751	5.850823
NHS11B0004905	5.667688

▼ Code

```
# Visualisation
plot(dev_i, type = "h", main = "Individual Deviances of ALT Mode
     xlab = "Index", ylab = "Deviance", ylim = c(0, max(dev_i) *
points(dev_i, pch = 19, col = "steelblue", cex = 1)
points(top.10.idx.alt, top.10.devs.alt, pch = 19, col = "darkred
```



▼ Code

```
# Outlier analysis for GGT Subpopulation Model
prob = predict(ggt.mlr, type = "probs")
```

```
obs = model.response(model.frame(ggt.mlr))
p_true = prob[cbind(seq_along(obs), match(obs, colnames(prob)))]
dev_i = -2 * log(p_true)

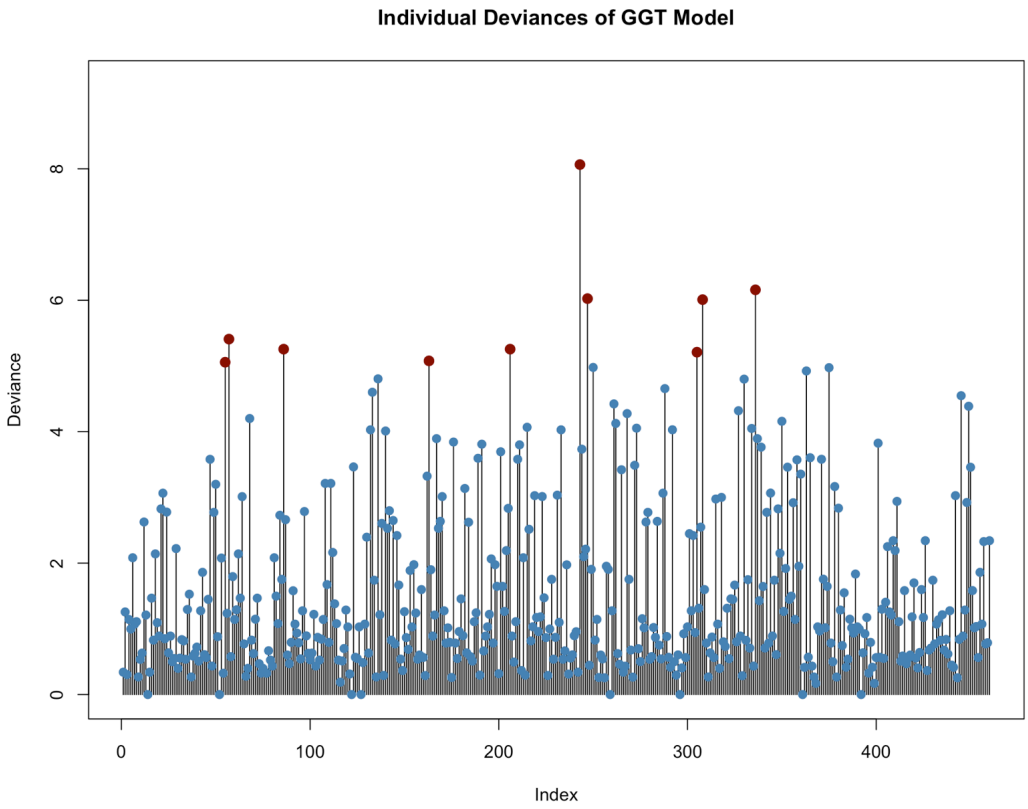
# Top 10 observations
top.10.idx.ggt = order(dev_i, decreasing = TRUE)[1:10]
top.10.devs.ggt = dev_i[top.10.idx.ggt]
top.10.outliers.ggt = data.frame("ABSLID" = pdat.full$ABSLID[top
                                 "Deviance" = top.10.devs.ggt)
kable(top.10.outliers.ggt, align = "cc", caption = "GGT Model T
```

GGT Model Top 10 Outliers

ABSLID	Deviance
NHS11B0002736	8.064951
NHS11B0003784	6.158953
NHS11B0002786	6.025021
NHS11B0003471	6.009697
NHS11B0000747	5.408706
NHS11B0000968	5.256003
NHS11B0002313	5.256003
NHS11B0003442	5.209729
NHS11B0001941	5.077694
NHS11B0000732	5.057100

▼ Code

```
# Visualisation
plot(dev_i, type = "h", main = "Individual Deviances of GGT Mode
     xlab = "Index", ylab = "Deviance", ylim = c(0, max(dev_i) *
points(dev_i, pch = 19, col = "steelblue", cex = 1)
points(top.10.idx.ggt, top.10.devs.ggt, pch = 19, col = "darkred
```



▼ Code

```
# Outlier analysis for ALT Full Population Model
prob = predict(alt.fmlr, type = "probs")
obs = model.response(model.frame(alt.fmlr))
p_true = prob[cbind(seq_along(obs), match(obs, colnames(prob)))]
dev_i = -2 * log(p_true)

# Top 10 observations
top.10.idx.altf = order(dev_i, decreasing = TRUE)[1:10]
top.10.devs.altf = dev_i[top.10.idx.altf]
top.10.outliers.altf = data.frame("ABSLID" = fdat.full$ABSLID[t
                                  "Deviance" = top.10.devs.altf)
kable(top.10.outliers.altf, align = "cc", caption = "ALT Full M
```

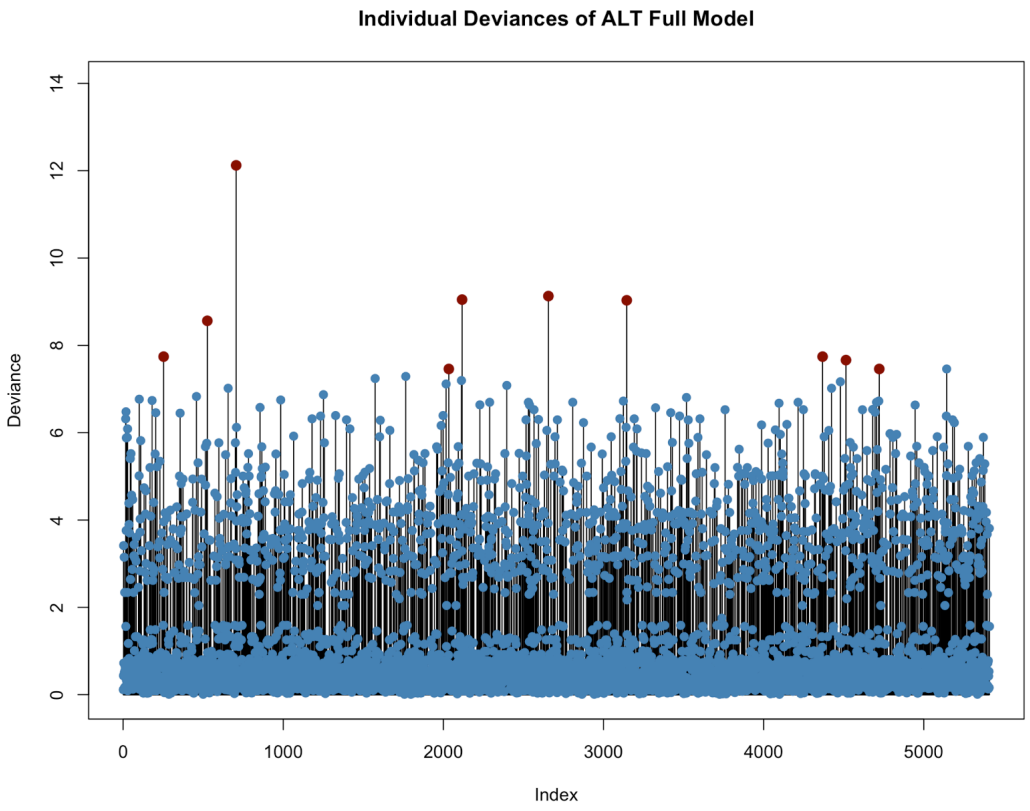
ALT Full Model Top 10 Outliers

ABSLID	Deviance
NHS11B0000713	12.122599
NHS11B0002669	9.128581
NHS11B0002128	9.047451
NHS11B0003159	9.032050

NHS11B0000530	8.562716
NHS11B0000256	7.741989
NHS11B0004391	7.741989
NHS11B0004537	7.662898
NHS11B0002045	7.459568
NHS11B0004747	7.459568

▼ Code

```
# Visualisation
plot(dev_i, type = "h", main = "Individual Deviances of ALT Ful
     xlab = "Index", ylab = "Deviance", ylim = c(0, max(dev_i) *
points(dev_i, pch = 19, col = "steelblue", cex = 1)
points(top.10.idx.altf, top.10.devs.altf, pch = 19, col = "dark
```



▼ Code

```
# Outlier analysis for GGT Full Population Model
prob = predict(ggt.fmlr, type = "probs")
obs = model.response(model.frame(ggt.fmlr))
p_true = prob[cbind(seq_along(obs), match(obs, colnames(prob)))]
dev_i = -2 * log(p_true)
```

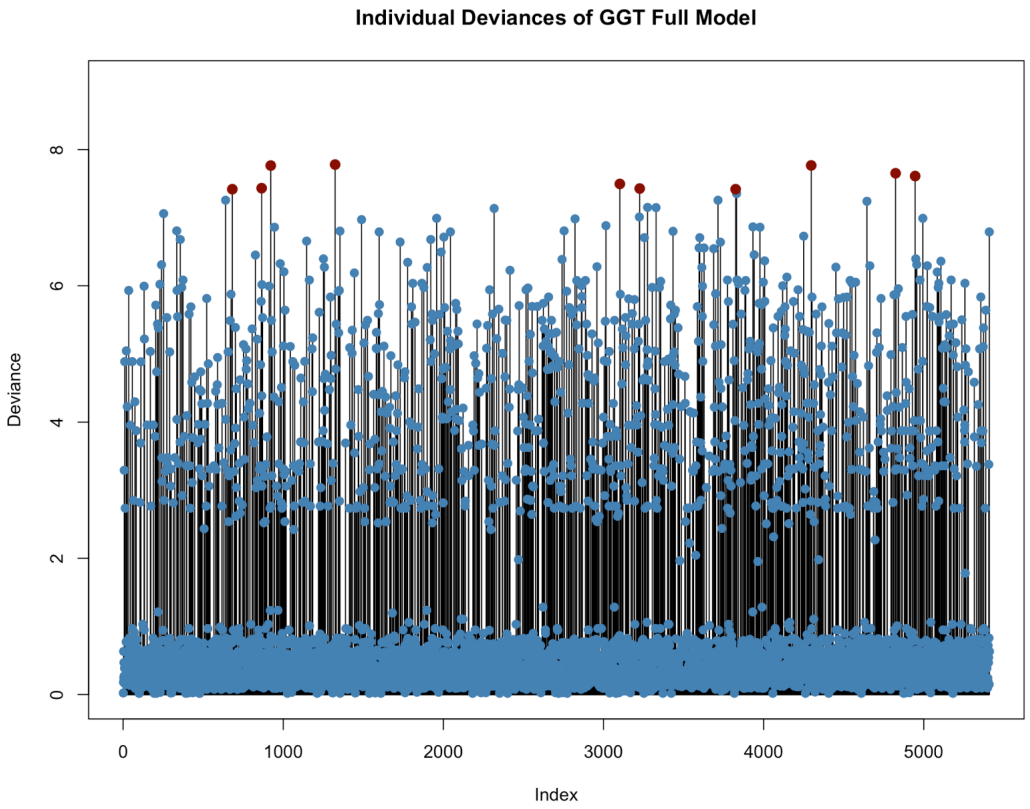
```
# Top 10 observations
top.10.idx.ggtf = order(dev_i, decreasing = TRUE)[1:10]
top.10.devs.ggtf = dev_i[top.10.idx.ggtf]
top.10.outliers.ggtf = data.frame("ABSLID" = fdat.full$ABSLID[t
                                  "Deviance" = top.10.devs.ggtf)
kable(top.10.outliers.ggtf, align = "cc", caption = "GGT Full M
```

GGT Full Model Top 10 Outliers

ABSLID	Deviance
NHS11B0001332	7.779955
NHS11B0004320	7.766957
NHS11B0000930	7.765078
NHS11B0004850	7.652125
NHS11B0004973	7.611300
NHS11B0003116	7.495640
NHS11B0000873	7.433604
NHS11B0003241	7.428302
NHS11B0000688	7.418022
NHS11B0003843	7.418022

▼ Code

```
# Visualisation
plot(dev_i, type = "h", main = "Individual Deviances of GGT Ful
     xlab = "Index", ylab = "Deviance", ylim = c(0, max(dev_i) >
points(dev_i, pch = 19, col = "steelblue", cex = 1)
points(top.10.idx.ggtf, top.10.devs.ggtf, pch = 19, col = "dark
```



Collinearity Analysis

We cannot use VIF for our models, as the standard functions do not work well from my testing. Instead, we use matrix methods to determine whether there is perfect multicollinearity, where we essentially check the rank of the QR factorisation of the design matrix $\mathbf{X}$ and compare that to the number of columns of $\mathbf{X}$. The condition number κ also measures how "close" we are to numerical multicollinearity; smaller values mean that the "multicollinearity" here is not as strong. The values below do show some moderate concern.

▼ Code

```
# Helper function
rk.collinear.check = function(model, model_name = NULL) {

  X = model.matrix(model)
  X_no_intercept = X[, colnames(X) != "(Intercept)", drop = FALS
  qr_rank = qr(X)$rank
  n_cols = ncol(X)

  data.frame(model = ifelse(is.null(model_name), "", model_name
             n_obs = nrow(X),
```

```
              n_columns = n_cols,
              qr_rank = qr_rank,
              rank_deficiency = n_cols - qr_rank,
              full_rank = qr_rank == n_cols,
              kappa = kappa(X_no_intercept))

}

# Calculating
col_alt = rk.collinear.check(alt.mlr, "ALT")
col_ggt = rk.collinear.check(ggt.mlr, "GGT")
col_altf = rk.collinear.check(alt.fmlr, "ALT Full")
col_ggtf = rk.collinear.check(ggt.fmlr, "GGT Full")

# Table
col.res = bind_rows(col_alt, col_ggt, col_altf, col_ggtf)
new.cols = c("Model", "# Obs", "# Columns", "QR Rank",
             "Rank Deficiency", "Full Rank", "Kappa")
kable(col.res, digits = 3, align = "cccccccc", col.names = new.
      caption = "Collinearity Checks for MLR Models; Check Full
```

Collinearity Checks for MLR Models; Check Full Rank = True

Model	# Obs	# Columns	QR Rank	Rank Deficiency	Full Rank	Kappa
ALT	460	22	21	1	FALSE	27.579
GGT	460	22	21	1	FALSE	27.579
ALT Full	5410	22	22	0	TRUE	26.033
GGT Full	5410	22	22	0	TRUE	26.033

It seems that the collinearity assumption is violated for the restricted models, which gives us some evidence that we may have to restrict our modelling to the full adult population.

We can check what is causing the rank deficiency as follows.

▼ Code

```
# ALT rank deficiency check
X = model.matrix(alt.mlr)
q = qr(X)
independent_cols = q$pivot[seq_len(q$rank)]
dependent_cols = q$pivot[seq.int(q$rank + 1, ncol(X))]
colnames(X)[dependent_cols]
```

```
[1] "AGEB85 years and older"
```

▼ Code

```
# GGT rank deficiency check
X = model.matrix(ggt.mlr)
q = qr(X)
independent_cols = q$pivot[seq_len(q$rank)]
dependent_cols = q$pivot[seq.int(q$rank + 1, ncol(X))]
colnames(X)[dependent_cols]
```

```
[1] "AGEB85 years and older"
```

In both cases, it is the last category of `AGEB`. This tells us we could either try checking if we can drop levels, or collapse the age variable at some point. Overall, this result gives another reason to be more cautious about coefficient estimates from the restricted population, which is consistent with the instability we have already been seeing there.

Sensitivity Analysis

We will take the top 10 outliers from above, and then measure their influence by removing them from our model data set, and see what happens to the `EXREGUIC` coefficients. In essence this helps us answer the question: Of the ten worst-fitting participants, which ones have the largest impact on the estimated association between exercise and ALT/GGT?

▼ Code

```
# Helper function
case.delete.influence = function(model, top_idx) {

  dat = model.frame(model)
  beta_original = coef(model)
  se_original = summary(model)$standard.errors

  results = lapply(top_idx, function(i) {

    fit_i = multinom(formula(model), data = dat[-i, , drop = FAI
                     Hess = TRUE, trace = FALSE)
    beta_i = coef(fit_i)

    # Ensure dimensions match
    change = beta_i - beta_original
    scaled_change = change / se_original
    data.frame(index = i,
```

```
                outcome = rep(rownames(beta_original), each = nc
                term = rep(colnames(beta_original), times = nrow
                original = as.vector(t(beta_original)),
                deleted = as.vector(t(beta_i)),
                absolute_change = as.vector(t(change)),
                percent_change = 100 * as.vector(t(change)) /
                  as.vector(t(beta_original)),
                scaled_change = as.vector(t(scaled_change))

    )

  })

  bind_rows(results)

}

# Applying to our models; take only EXREGUIC
new.cols = c("Index", "Outcome", "Original", "Deleted", "Abs Cha
             "% Change", "Scaled Change")
infl.alt = case.delete.influence(alt.mlr, top.10.idx.alt) %>%
  filter(grepl("^EXREGUIC", term)) %>% arrange(index) %>%
  select(-term)
kable(infl.alt, digits = 3,
      caption = "Influence Measures for ALT on EXREGUIC (Did Not
      align = "cccccccc", col.names = new.cols)
```

Influence Measures for ALT on EXREGUIC (Did Not Meet Recommended Guidelines)

Index	Outcome	Original	Deleted	Abs Change	% Change	Scaled Change
47	Borderline	-0.707	-0.704	0.003	-0.416	0.009
47	High	-0.242	-0.229	0.013	-5.503	0.045
58	Borderline	-0.707	-0.703	0.004	-0.531	0.011
58	High	-0.242	-0.230	0.012	-5.156	0.042
103	Borderline	-0.707	-0.720	-0.013	1.902	-0.039
103	High	-0.242	-0.316	-0.073	30.262	-0.248
133	Borderline	-0.707	-0.745	-0.038	5.418	-0.111
133	High	-0.242	-0.246	-0.004	1.556	-0.013
163	Borderline	-0.707	-0.797	-0.091	12.812	-0.263
163	High	-0.242	-0.258	-0.016	6.518	-0.053
210	Borderline	-0.707	-0.783	-0.077	10.835	-0.222

210	High	-0.242	-0.260	-0.018	7.368	-0.060
279	Borderline	-0.707	-0.715	-0.009	1.264	-0.026
279	High	-0.242	-0.292	-0.050	20.474	-0.168
342	Borderline	-0.707	-0.789	-0.083	11.679	-0.240
342	High	-0.242	-0.255	-0.012	5.125	-0.042
370	Borderline	-0.707	-0.778	-0.072	10.180	-0.209
370	High	-0.242	-0.247	-0.005	1.908	-0.016
442	Borderline	-0.707	-0.703	0.004	-0.514	0.011
442	High	-0.242	-0.226	0.016	-6.634	0.054

▼ Code

```
infl.ggt = case.delete.influence(ggt.mlr, top.10.idx.ggt) %>%
  filter(grepl("^EXREGUIC", term)) %>% arrange(index) %>%
  select(-term)
kable(infl.ggt, digits = 3,
      caption = "Influence Measures for GGT on EXREGUIC (Did Not
      align = "cccccccc", col.names = new.cols)
```

Influence Measures for GGT on EXREGUIC (Did Not Meet Recommended Guidelines)

Index	Outcome	Original	Deleted	Abs Change	% Change	Scaled Change
55	Borderline	-0.078	-0.071	0.007	-8.988	0.020
55	High	0.182	0.157	-0.025	-13.837	-0.103
57	Borderline	-0.078	-0.153	-0.075	96.024	-0.211
57	High	0.182	0.183	0.002	0.924	0.007
86	Borderline	-0.078	-0.023	0.055	-70.833	0.156
86	High	0.182	0.179	-0.002	-1.357	-0.010
163	Borderline	-0.078	-0.139	-0.061	78.286	-0.172
163	High	0.182	0.183	0.001	0.479	0.004
206	Borderline	-0.078	-0.023	0.055	-70.833	0.156
206	High	0.182	0.179	-0.002	-1.357	-0.010
243	Borderline	-0.078	-0.154	-0.075	96.363	-0.212
243	High	0.182	0.187	0.005	2.798	0.021
247	Borderline	-0.078	-0.017	0.061	-77.862	0.171
247	High	0.182	0.183	0.002	0.978	0.007

305	Borderline	-0.078	-0.079	0.000	0.505	-0.001
305	High	0.182	0.147	-0.034	-18.820	-0.140
308	Borderline	-0.078	-0.080	-0.001	1.819	-0.004
308	High	0.182	0.160	-0.021	-11.756	-0.088
336	Borderline	-0.078	-0.117	-0.038	49.056	-0.108
336	High	0.182	0.181	0.000	-0.200	-0.001

▼ Code

```
infl.altf = case.delete.influence(alt.fmlr, top.10.idx.altf) %>%
  filter(grepl("^EXREGUIC", term)) %>% arrange(index) %>%
  select(-term)
kable(infl.altf, digits = 3,
      caption = "Influence Measures for ALT Full on EXREGUIC (Di
      align = "cccccccc", col.names = new.cols)
```

Influence Measures for ALT Full on EXREGUIC (Did Not Meet Recommended Guidelines)

Index	Outcome	Original	Deleted	Abs Change	% Change	Scaled Change
253	Borderline	-0.039	-0.040	-0.001	1.615	-0.007
253	High	0.213	0.209	-0.004	-1.900	-0.045
526	Borderline	-0.039	-0.043	-0.004	10.796	-0.046
526	High	0.213	0.212	-0.001	-0.320	-0.008
706	Borderline	-0.039	-0.038	0.001	-2.046	0.009
706	High	0.213	0.218	0.005	2.456	0.058
2034	Borderline	-0.039	-0.039	0.000	-1.034	0.004
2034	High	0.213	0.216	0.003	1.591	0.038
2117	Borderline	-0.039	-0.035	0.004	-9.440	0.040
2117	High	0.213	0.213	0.001	0.267	0.006
2656	Borderline	-0.039	-0.044	-0.005	11.869	-0.050
2656	High	0.213	0.212	-0.001	-0.360	-0.009
3145	Borderline	-0.039	-0.040	0.000	1.063	-0.005
3145	High	0.213	0.210	-0.002	-1.139	-0.027
4368	Borderline	-0.039	-0.040	-0.001	1.615	-0.007
4368	High	0.213	0.209	-0.004	-1.900	-0.045
4514	Borderline	-0.039	-0.039	0.000	-0.509	0.002

4514	High	0.213	0.210	-0.003	-1.521	-0.036
4722	Borderline	-0.039	-0.039	0.000	-1.034	0.004
4722	High	0.213	0.216	0.003	1.591	0.038

▼ Code

```
infl.ggtf = case.delete.influence(ggt.fmlr, top.10.idx.ggtf) %>%
  filter(grepl("^EXREGUIC", term)) %>% arrange(index) %>%
  select(-term)
kable(infl.ggtf, digits = 3,
      caption = "Influence Measures for GGT Full on EXREGUIC (D
      align = "cccccccc", col.names = new.cols)
```

Influence Measures for GGT Full on EXREGUIC (Did Not Meet Recommended Guidelines)

Index	Outcome	Original	Deleted	Abs Change	% Change	Scaled Change
682	Borderline	0.164	0.170	0.006	3.716	0.049
682	High	0.361	0.361	0.000	0.072	0.003
865	Borderline	0.164	0.170	0.006	3.799	0.051
865	High	0.361	0.361	0.000	0.058	0.002
922	Borderline	0.164	0.170	0.006	3.690	0.049
922	High	0.361	0.361	0.000	0.077	0.003
1324	Borderline	0.164	0.171	0.007	4.141	0.055
1324	High	0.361	0.361	0.000	0.104	0.004
3102	Borderline	0.164	0.164	0.000	-0.262	-0.003
3102	High	0.361	0.356	-0.005	-1.318	-0.056
3226	Borderline	0.164	0.154	-0.010	-6.284	-0.084
3226	High	0.361	0.360	-0.001	-0.179	-0.008
3825	Borderline	0.164	0.170	0.006	3.716	0.049
3825	High	0.361	0.361	0.000	0.072	0.003
4297	Borderline	0.164	0.170	0.006	3.681	0.049
4297	High	0.361	0.361	0.000	0.047	0.002
4824	Borderline	0.164	0.169	0.005	2.972	0.040
4824	High	0.361	0.361	0.000	0.082	0.003
4946	Borderline	0.164	0.171	0.007	4.270	0.057
4946	High	0.361	0.361	0.000	0.074	0.003

It may be easier to visualise the changes.

▼ Code

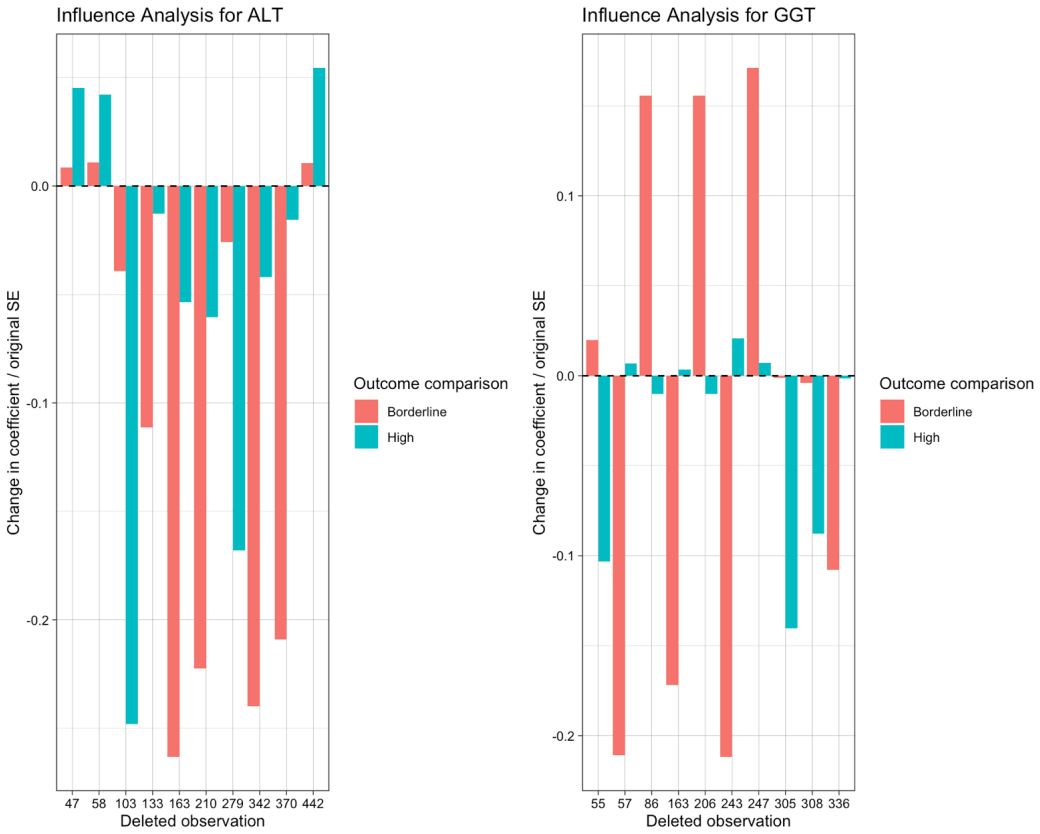
```
# Visualise the differences
p1 = ggplot(infl.alt, aes(x = factor(index), y = scaled_change,
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = 2) +
  labs(x = "Deleted observation", y = "Change in coefficient / (
       fill = "Outcome comparison", title = "Influence Analysis
  theme_linedraw()

p2 = ggplot(infl.ggt, aes(x = factor(index), y = scaled_change,
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = 2) +
  labs(x = "Deleted observation", y = "Change in coefficient / (
       fill = "Outcome comparison", title = "Influence Analysis
  theme_linedraw()

p3 = ggplot(infl.altf, aes(x = factor(index), y = scaled_change,
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = 2) +
  labs(x = "Deleted observation", y = "Change in coefficient / (
       fill = "Outcome comparison", title = "Influence Analysis
  theme_linedraw()

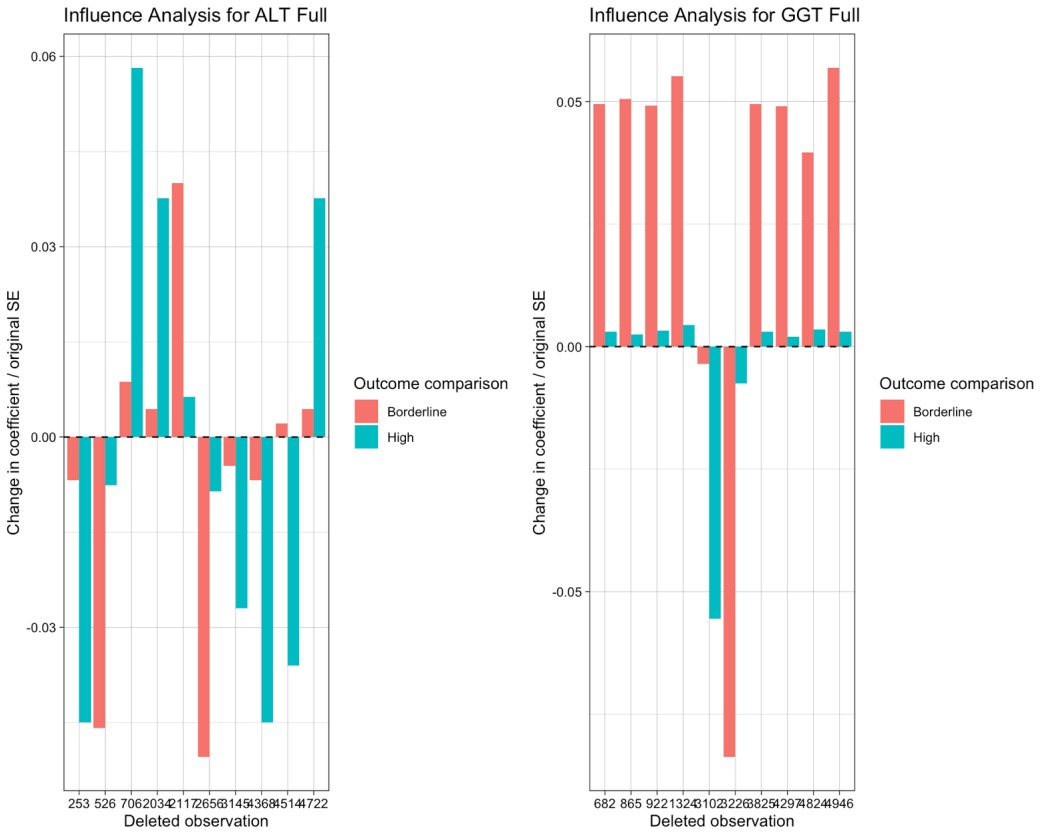
p4 = ggplot(infl.ggtf, aes(x = factor(index), y = scaled_change,
  geom_col(position = "dodge") +
  geom_hline(yintercept = 0, linetype = 2) +
  labs(x = "Deleted observation", y = "Change in coefficient / (
       fill = "Outcome comparison", title = "Influence Analysis
  theme_linedraw()

p1 + p2
```


Influence Analysis for ALT
Change in coefficient / original SE
0.0
-0.1
-0.2
47 58 103 133 163 210 279 342 370 442
Deleted observation
Outcome comparison
Borderline
High
Influence Analysis for GGT
Change in coefficient / original SE
0.1
0.0
-0.1
-0.2
55 57 86 163 206 243 247 305 308 336
Deleted observation
Outcome comparison
Borderline
High

▼ Code

```
p3 + p4
```



Model Evaluation

We will use the Hand-Till multiclass AUC and log-loss to essentially compare the ALT model under the subpopulation against the ALT model under the full population, and similarly for GGT. This helps us answer: does the model discriminate/predict liver-enzyme category differently in the restricted population versus the broader adult population?

Multi-class AUC

We use the `pROC::multiclass.roc` implementation to calculate the AUCROC, and then return the results as a clean data frame.

▼ Code

```
# Helper function
get_multiclass_auc = function(model, model_name = NULL) {

  obs = model.response(model.frame(model))
  prob = predict(model, type = "probs")
```

```
  roc_obj = multiclass.roc(response = obs, predictor = prob)
  data.frame(model = ifelse(is.null(model_name), "", model_name
             multiclass_auc = as.numeric(roc_obj$auc))

}

auc.results = bind_rows(get_multiclass_auc(alt.mlr, "ALT"),
                        get_multiclass_auc(ggt.mlr, "GGT"),
                        get_multiclass_auc(alt.fmlr, "ALT Full"
                        get_multiclass_auc(ggt.fmlr, "GGT Full"
```

Log Loss

Log-loss is essentially the average negative log probability assigned to the truth. It is one of the recommended scores to compare classification algorithms, so we use it here.

▼ Code

```
# Helper function
get_log_loss = function(model, model_name = NULL) {

  obs = model.response(model.frame(model))
  prob = predict(model, type = "probs")
  true_class_col = match(obs, colnames(prob))
  p_true = prob[cbind(seq_along(obs), true_class_col)]
  log_loss = -mean(log(p_true))

  data.frame(model = ifelse(is.null(model_name), "", model_name
             log_loss = log_loss)

}

LL.results = bind_rows(get_log_loss(alt.mlr, "ALT"),
                       get_log_loss(ggt.mlr, "GGT"),
                       get_log_loss(alt.fmlr, "ALT Full"),
                       get_log_loss(ggt.fmlr, "GGT Full"))
```

Overall Evaluation

We combine the two tables for easy comparison between the restricted and full models

▼ Code

```
eval.tab = inner_join(auc.results, LL.results, by = "model")
eval.tab.sorted = eval.tab[c(1, 3, 2, 4), ]

kable(eval.tab.sorted, caption = "AUCROC and Log Loss Results fo
      align = "cc",
      col.names = c("Model", "Multiclass AUC", "Log Loss"), digi
```

AUCROC and Log Loss Results for Our Models

	Model	Multiclass AUC	Log Loss
1	ALT	0.7150	0.6608
3	ALT Full	0.6553	0.6417
2	GGT	0.6789	0.7609
4	GGT Full	0.6200	0.5651

The multinomial models showed modest in-sample discriminatory ability, with multiclass AUC values ranging from 0.620 to 0.715. The restricted-population models had higher AUCs for both ALT and GGT, although the full-population models had lower log loss. As AUC and log loss assess different aspects of model performance, and the restricted and full models were fitted to different populations with potentially different class distributions, these measures should not be interpreted as a direct ranking of model quality. Furthermore, the metrics are calculated in-sample and therefore describe model fit to the observed data rather than performance on unseen participants.

In other words, the models have some ability to distinguish liver-enzyme categories, but they are not highly predictive classifiers; these results are mainly a diagnostic check rather than evidence that the models could accurately classify new individuals.